# High-Level Design (HLD)

## AI-Powered Security Analytics Platform

| Field | Value |
|---|---|
| **Document ID** | HLD-001 |
| **Version** | 1.0 |
| **Date** | 2026-07-18 |
| **Status** | Draft |
| **Architecture Reference** | ADR-001 |
| **Requirements Reference** | SRS-001 |

## Table of Contents


1. Overall Architecture
2. Component Diagram
3. Data Flow
4. Module Responsibilities
5. Technology Stack
6. Communication Between Modules


## 1. Overall Architecture

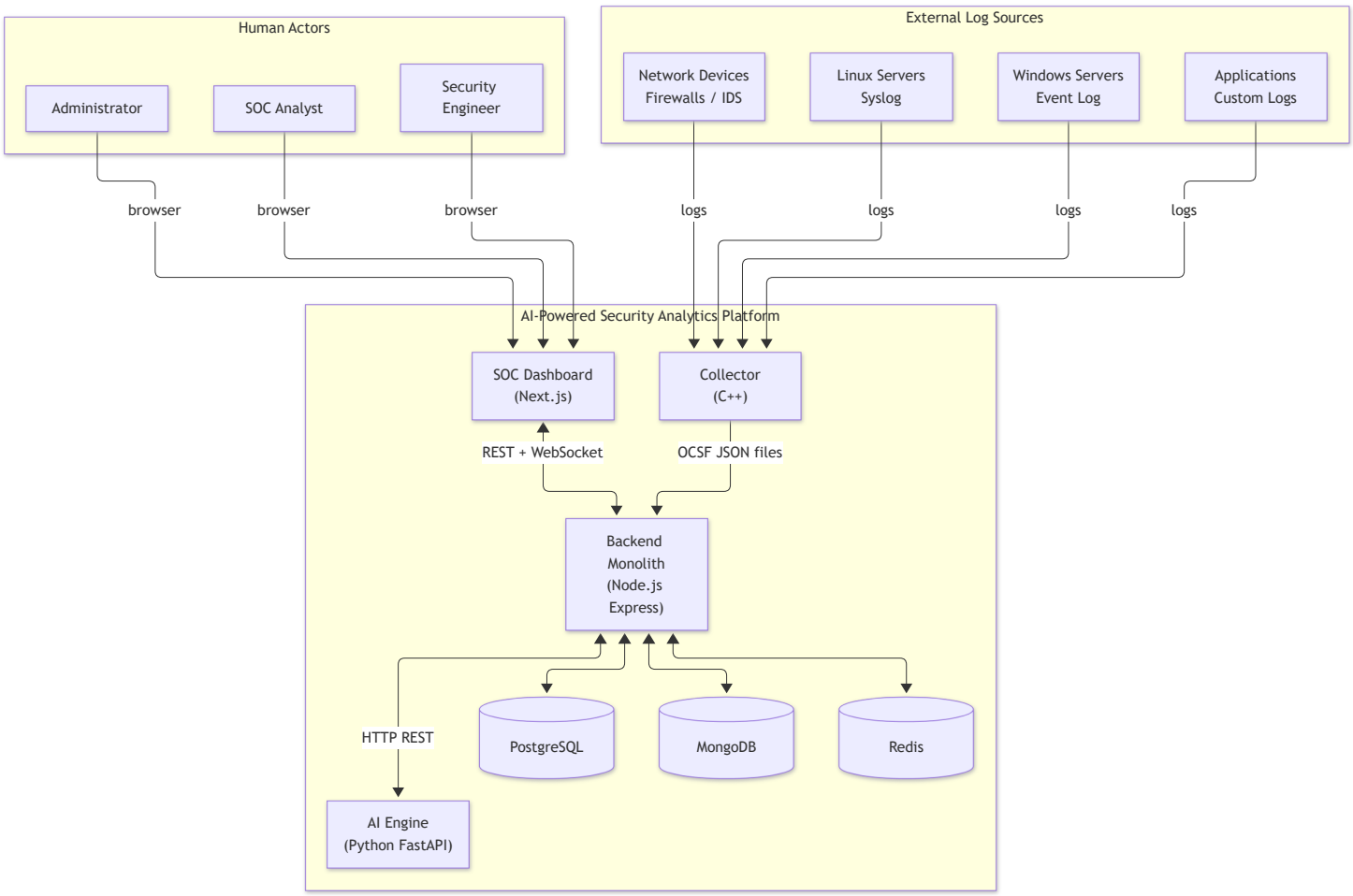
The AI-Powered Security Analytics Platform follows a **Modular Monolith** architecture. The system is composed of **4 independent processes** that communicate through **file-based I/O**, **HTTP REST**, and **WebSocket**. There are no distributed message brokers, no service meshes, and no container orchestration.

# 1.1 Architectural Style

| Property | Value |
|---|---|
| **Style** | Modular Monolith |
| **Design Principles** | Clean Architecture, SOLID, Repository Pattern, Dependency Injection |
| **Processes** | 4 (Collector, Backend, AI Engine, Frontend) |
| **Databases** | 3 (PostgreSQL, MongoDB, Redis) |
| **Deployment** | Docker Compose (single command) |
| **Tenancy** | Single-tenant (MVP) |

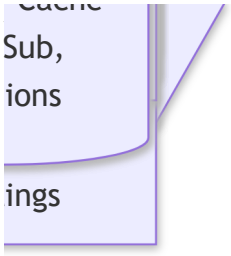
# 1.2 System Context Diagram

This diagram shows the platform in context with its external actors and the environment it operates within.



## 1.3 High-Level Architecture Diagram

This diagram details the internal structure of each process and how they interconnect.




# 2. Component Diagram

## 2.1 Process-Level Component Diagram

This diagram shows the four processes and the three data stores as top-level components, with their interfaces and dependencies.

--- id: ed1ced68-30ae-4bf6-83ae-1b5ae6e8da6d --- graph LR subgraph "Collector [C++]" C_IN(("Input\nSyslog / File / ETW / HTTP")) C_OUT(("Output\nOCSF JSON Files")) C_HB(("Output\nHeartbeat Files")) end subgraph "Backend [Node.js Express]" B_FS(("Input\nFile System Watch")) B_AI(("Output\nHTTP to AI Engine")) B_API(("Interface\nREST API")) B_WS(("Interface\nWebSocket")) B_DB(("Output\nDB Read/Write")) end subgraph "AI Engine [Python FastAPI]" AI_API(("Interface\nREST API")) end subgraph "Frontend [Next.js]" FE_REST(("Output\nREST Client")) FE_WS(("Input\nWebSocket Client")) end PG[(PostgreSQL)] MDB[(MongoDB)] RD[(Redis)] C_OUT -->|OCSF .json files| B_FS C_HB -->|heartbeat files| B_FS B_AI -->|HTTP POST /api/v1/*| AI_API AI_API -->|JSON response| B_AI FE_REST -->|HTTP GET/POST/PUT/DELETE| B_API B_WS -->|real-time events| FE_WS B_DB --> PG & MDB & RD

## 2.2 Backend Internal Component Diagram

The backend monolith is organized into **9 modules**, a **workers layer**, and a **shared kernel**. Each module follows Clean Architecture with 4 layers: Domain, Application, Infrastructure, Interface.

graph TB subgraph "Backend Monolith — Internal Components" subgraph "Entry Points" EP_HTTP["Express HTTP Server<br/>(REST API)"] EP_WS["WebSocket Server<br/>(Real-time)"] EP_FS["File System Watcher<br/>(chokidar)"] end subgraph "Modules (Vertical Slices)"

M_ING["Ingestion Module<br/>Directory Watcher<br/>IProcessingQueue"] M_PAR["Parsing Module<br/>Multi-format Parser<br/>OCSF Normalizer"] M_ANA["Analysis Module<br/>Feature Extractor<br/>Rule Engine<br/>AI Client"] M_COR["Correlation Module<br/>Incident Correlator<br/>Risk Scorer"] M_INC["Incidents Module<br/>CRUD + Lifecycle<br/>Status Transitions"] M_RUL["Rules Module<br/>Sigma Rule CRUD<br/>Import / Export"] M_AUTH["Auth Module<br/>JWT + RBAC<br/>Audit Logging"] M_CMON["Collector Monitoring<br/>Heartbeat Tracking<br/>Queue Metrics"] M_DASH["Dashboard Module<br/>Aggregations<br/>Statistics"] end subgraph "Workers" W_PIPE["Pipeline Worker<br/>Dequeue → Parse → Normalize<br/>→ Detect → Correlate → Score"] end subgraph "Shared Kernel" SK_OCSF["OCSF Schema Defs"] SK_ERR["Error Handling"] SK_LOG["Logger"] SK_CFG["Configuration"] end subgraph "Repository Interfaces (Domain)" RI_INC{{"IIncidentRepository"}} RI_LOG{{"ILogRepository"}} RI_RUL{{"IRuleRepository"}} RI_USR{{"IUserRepository"}} RI_AUD{{"IAuditRepository"}} RI_COL{{"ICollectorStatusRepository"}} end subgraph "Repository Implementations (Infrastructure)" IMPL_PG["PostgreSQL Repositories<br/>(pg / Knex)"] IMPL_MG["MongoDB Repositories<br/>(Mongoose)"] IMPL_RD["Redis Repositories<br/>(ioredis)"] end end EP_FS --> M_ING M_ING --> W_PIPE W_PIPE --> M_PAR --> M_ANA --> M_COR M_COR --> RI_INC & RI_LOG EP_HTTP --> M_INC & M_RUL & M_AUTH & M_CMON & M_DASH EP_WS --> M_DASH M_INC --> RI_INC M_RUL --> RI_RUL M_AUTH --> RI_USR & RI_AUD M_CMON --> RI_COL M_DASH --> RI_INC & RI_LOG RI_INC & RI_AUD & RI_RUL & RI_USR & RI_COL --> IMPL_PG RI_LOG --> IMPL_MG M_ING --> IMPL_RD style W_PIPE fill:#8e44ad,color:#fff style SK_OCSF fill:#2c3e50,color:#fff

## 2.3 AI Engine Component Diagram

graph TB subgraph "AI Engine — Python FastAPI" ROUTER[FastAPI Router] subgraph "Endpoints" EP_ANOM["POST /api/v1/detect/anomaly"] EP_EXPL["POST /api/v1/explain"] EP_CLAS["POST /api/v1/classify/threat<br/>(optional)"] EP_HLTH["GET /api/v1/health"] end subgraph "ML Layer" IF_MODEL["Isolation Forest Model<br/>(scikit-learn)"] SHAP_ENG["SHAP Explainer<br/>(TreeExplainer)"] TC_MODEL["Threat Classifier<br/>(Random Forest / XGBoost)<br/>(optional)"] end subgraph "Infrastructure" PREPROC["Feature Preprocessor<br/>Scaling / Encoding"] ML_LOAD["Model Loader<br/>joblib / ONNX<br/>Version from config.json"] SCHEMAS["Pydantic Schemas<br/>Request / Response validation"] end end ROUTER --> EP_ANOM & EP_EXPL & EP_CLAS & EP_HLTH EP_ANOM --> PREPROC --> IF_MODEL EP_EXPL --> SHAP_ENG EP_CLAS --> PREPROC --> TC_MODEL IF_MODEL -.->|loaded at startup| ML_LOAD TC_MODEL -.->|loaded at startup| ML_LOAD SHAP_ENG -.->|wraps| IF_MODEL

## 2.4 Collector Component Diagram

graph TB subgraph "Collector — C++" subgraph "Input Sources" S_SYS["Syslog Receiver<br/>UDP/TCP (RFC 5424)"] S_FILE["File Tailer<br/>inotify / ReadDirectoryChanges"] S_ETW["ETW Reader<br/>Windows Event Log"] S_HTTP["HTTP Receiver<br/>JSON endpoint"] end subgraph "Core Pipeline" OCSF_CONV["OCSF Converter<br/>Source → OCSF JSON mapping"] BATCHER["Batcher<br/>Count-based + Time-based"] ATOM_WR["Atomic Writer<br/>.tmp → .json rename"] end subgraph "Support" CHKPT["Checkpoint Manager<br/>Track read positions"] HB_WR["Heartbeat Writer<br/>Periodic status output"] CONF["Config Parser<br/>YAML configuration"] end end OUTDIR[(Collector Directory)] S_SYS & S_FILE & S_ETW & S_HTTP --> OCSF_CONV OCSF_CONV --> BATCHER --> ATOM_WR --> OUTDIR S_SYS & S_FILE & S_ETW -.->|position updates| CHKPT HB_WR -->|periodic| OUTDIR CONF -.->|startup config| S_SYS & S_FILE & S_ETW & S_HTTP & BATCHER & ATOM_WR & HB_WR style OUTDIR fill:#27ae60,color:#fff

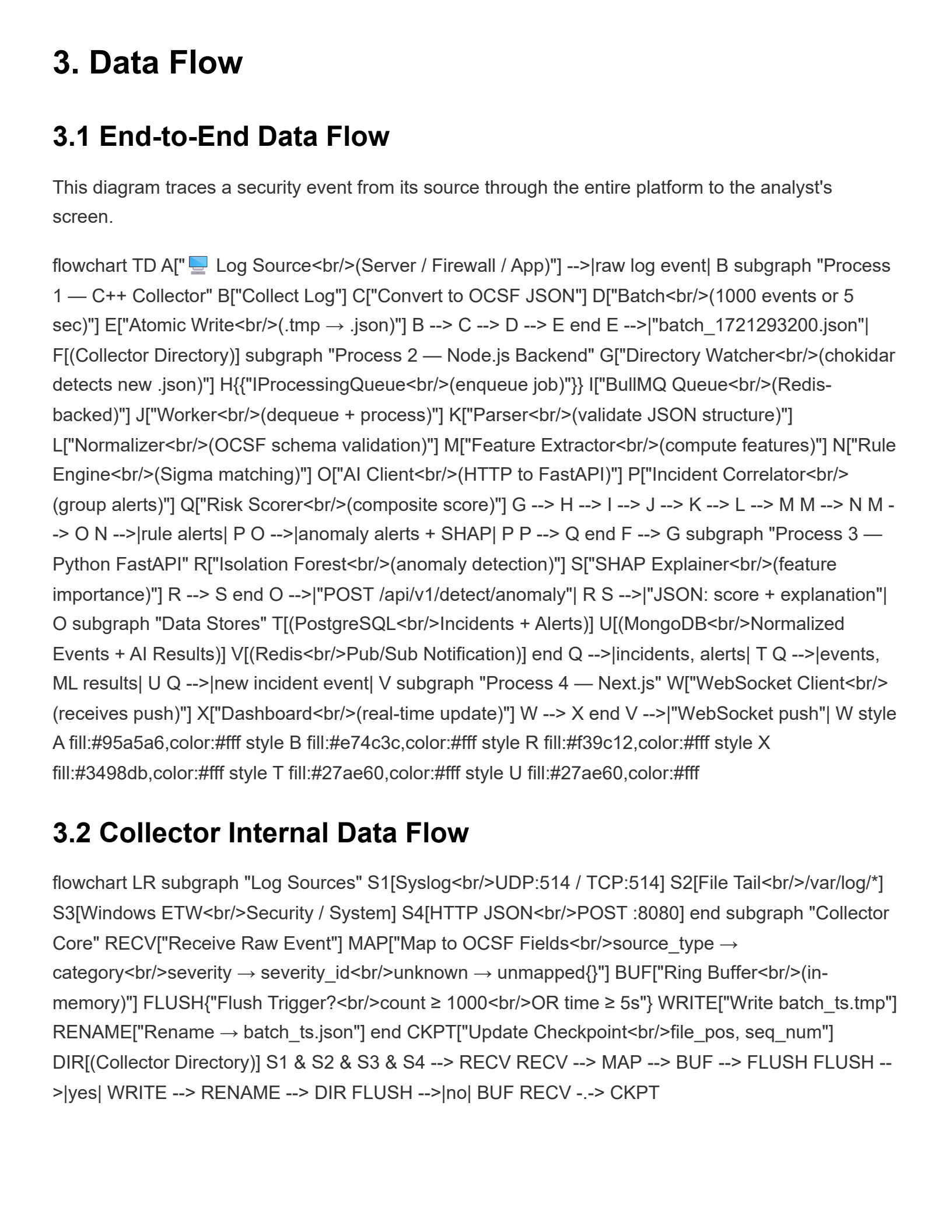
## 2.5 Frontend Component Diagram

graph TB subgraph "Frontend — Next.js" subgraph "Pages (App Router)" P_DASH["/ — Dashboard<br/>Real-time overview"] P_INC["/incidents — Incidents<br/>List + Detail"] P_INV["/investigate — Investigation<br/>Log search + Event timeline"] P_RUL["/rules — Rules<br/>Sigma editor + Testing"] P_AI["/ai-insights — AI Insights<br/>SHAP visualizations"] P_CMON["/collector — Collector Monitor<br/>Health + Metrics"] P_SET["/settings — Settings<br/>Users + Config"] P_AUD["/audit — Audit Log<br/>Action history"] P_LOGIN["/login — Authentication"] end subgraph "Shared Components" C_CHART["Chart Components<br/>Line / Bar / Pie / Heatmap"] C_TABLE["Data Table<br/>Sortable / Filterable / Paginated"] C_EDITOR["YAML Editor<br/>Sigma rule editing"] C_SHAP["SHAP Visualizer<br/>Feature importance display"] C_STATUS["Status Badge<br/>Online / Offline / Severity"] C_LAYOUT["Layout Shell<br/>Sidebar + Header + Content"] end subgraph "Services Layer" SVC_REST["REST API Client<br/>Axios / Fetch wrapper"] SVC_WS["WebSocket Client<br/>Real-time subscription"] SVC_AUTH["Auth Service<br/>JWT management"] end subgraph "State Management" STORE["Global Store<br/>Zustand / Context"] end end P_DASH & P_INC & P_INV & P_RUL & P_AI & P_CMON & P_SET & P_AUD --> C_LAYOUT P_DASH --> C_CHART & C_TABLE P_INC --> C_TABLE & C_STATUS P_RUL --> C_EDITOR P_AI --> C_SHAP P_CMON --> C_STATUS & C_TABLE & C_CHART P_DASH & P_INC --> SVC_WS P_INC & P_INV & P_RUL & P_CMON & P_SET & P_AUD --> SVC_REST P_LOGIN --> SVC_AUTH SVC_REST & SVC_WS & SVC_AUTH --> STORE

# 3. Data Flow

## 3.1 End-to-End Data Flow

This diagram traces a security event from its source through the entire platform to the analyst's screen.




## 3.2 Collector Internal Data Flow

## 3.3 Backend Pipeline Data Flow (Worker Detail)

flowchart TD JOB["Dequeued Job<br/>{filePath, timestamp, retryCount}"] JOB --> READ["Read File<br/>fs.readFile(filePath)"] READ --> PARSE["Parse JSON<br/>Extract event array"] PARSE -->|success| VALIDATE["Validate OCSF Schema<br/>per event"] PARSE -->|failure| QUARANTINE["Move to quarantine/<br/>Log parse error"] VALIDATE -->|valid events| FEAT["Extract Features<br/>• IP entropy<br/>• Login frequency<br/>• Time deviation<br/>• Byte volume<br/>• Process rarity"] VALIDATE -->|invalid events| FLAG["Flag + Store with<br/>validation_errors[]"] FEAT --> RULE["Rule Engine<br/>Evaluate against Sigma rules"] FEAT --> AIHTTP["HTTP POST → AI Engine<br/>/api/v1/detect/anomaly<br/>Timeout: 5 seconds"] AIHTTP -->|response| AI_RESULT["AI Result<br/>{anomaly_score, shap_values}"] AIHTTP -->|timeout / error| FALLBACK["Fallback<br/>Rule-only detection"] RULE -->|matches| MERGE["Merge Alerts<br/>Rule alerts + AI alerts"] AI_RESULT --> MERGE FALLBACK --> MERGE MERGE --> CORRELATE["Correlate Incidents<br/>Group by entity + time window"] CORRELATE --> SCORE["Risk Score<br/>rule_weight × ML_confidence × asset_criticality"] SCORE --> STORE_PG["Store Incident → PostgreSQL"] SCORE --> STORE_MDB["Store Events → MongoDB"] SCORE --> NOTIFY["Publish → Redis Pub/Sub"] SCORE --> CLEANUP["Delete processed file<br/>or move to processed/"] style JOB fill:#8e44ad,color:#fff style QUARANTINE fill:#e74c3c,color:#fff style FALLBACK fill:#e67e22,color:#fff

## 3.4 Authentication & Authorization Flow

sequenceDiagram actor Analyst as SOC Analyst participant FE as Next.js Frontend participant API as Express API participant Auth as Auth Module participant Redis as Redis participant PG as PostgreSQL Analyst->>FE: Enter credentials FE->>API: POST /api/v1/auth/login API->>Auth: authenticate(username, password) Auth->>PG: SELECT user WHERE username = ? PG-->>Auth: User record (bcrypt hash) Auth->>Auth: bcrypt.compare(password, hash) alt Valid credentials Auth->>Auth: Generate JWT (userId, role, exp) Auth->>Redis: Store session (JWT ID → userId) Auth-->>API: {token, user, role} API-->>FE: 200 OK + JWT FE->>FE: Store JWT in httpOnly cookie Note over Analyst, PG: Subsequent Requests Analyst->>FE: Navigate to /incidents FE->>API: GET /api/v1/incidents<br/>Authorization: Bearer <JWT> API->>Auth: verifyToken(JWT) Auth->>Redis: Check session exists Redis-->>Auth: Session valid Auth->>Auth: Check RBAC permission Auth-->>API: Authorized API->>PG: Query incidents PG-->>API: Incident records API-->>FE: 200 OK + incidents JSON FE-->>Analyst: Render incidents page else Invalid credentials Auth-->>API: Authentication failed API-->>FE: 401 Unauthorized FE-->>Analyst: Show error message end

## 3.5 Real-Time Dashboard Update Flow

sequenceDiagram participant Worker as Pipeline Worker participant PG as PostgreSQL participant Redis as Redis participant WS as WebSocket Server participant FE as Next.js Dashboard Note over FE: On page load, dashboard establishes WebSocket connection FE->>WS: Connect WebSocket WS->>Redis: SUBSCRIBE "incidents:new" Note over Worker: Pipeline detects threat and creates incident Worker->>PG: INSERT incident (severity=HIGH) Worker->>Redis: PUBLISH "incidents:new" {id, severity, title} Redis-->>WS: Message on "incidents:new" WS-->>FE: WebSocket push {type: "NEW_INCIDENT", data: {...}} FE->>FE: Update dashboard counters FE->>FE: Add incident to live feed FE->>FE: Flash notification badge

# 4. Module Responsibilities

## 4.1 Collector (C++) — Process 1

| Module | Responsibility | Inputs | Outputs |
|---|---|---|---|
| **Syslog Receiver** | Listen on UDP/TCP port 514, receive RFC 5424 syslog messages | Network packets | Raw syslog strings |
| **File Tailer** | Monitor log files for new lines, track read position | File system | Raw log lines |
| **ETW Reader** | Subscribe to Windows Event Log channels via ETW | Windows Event Tracing | Structured event records |
| **HTTP Receiver** | Accept JSON-formatted log events via REST endpoint | HTTP POST requests | JSON objects |
| **OCSF Converter** | Map source-specific fields to OCSF schema; place unknown fields in `unmapped` | Raw events from any source | OCSF JSON objects |
| **Batcher** | Accumulate events until batch threshold (count or time) is reached | Individual OCSF events | Array of OCSF events |
| **Atomic Writer** | Write batch to `.tmp` file, then rename to `.json` for atomic visibility | Batch array | File in collector directory |

| Module | Responsibility | Inputs | Outputs |
|---|---|---|---|
| **Checkpoint Manager** | Persist file read positions and sequence numbers for crash recovery | Read positions from sources | Checkpoint file on disk |
| **Heartbeat Writer** | Periodically write health status (timestamp, CPU, memory, events collected) | Internal metrics | Heartbeat file in collector directory |

# 4.2 Backend (Node.js Express) — Process 2

## 4.2.1 Ingestion Module

| Component | Responsibility |
|---|---|
| **Directory Watcher** | Monitors collector directory using chokidar; detects new `.json` files; enqueues jobs into `IProcessingQueue` |
| **IProcessingQueue** | Abstract interface for the processing queue. Methods: `enqueue()`, `dequeue()`, `retry()`, `deadLetter()`, `getStatus()` |
| **BullMQQueue** | Infrastructure implementation of `IProcessingQueue` using BullMQ (Redis-backed). Handles persistence, retry logic, priority, and dead-letter |

## 4.2.2 Workers

| Component | Responsibility |
|---|---|
| **Pipeline Worker** | Dequeues jobs from `IProcessingQueue` independently of Directory Watcher. Orchestrates the full processing pipeline: parse → normalize → extract features → rule engine + AI → correlate → score → store. Multiple workers run concurrently (configurable: 1–16) |

## 4.2.3 Parsing Module

| Component | Responsibility |
|---|---|
| **Parser** | Reads batch file content, parses JSON, validates structure. Detects format (OCSF JSON expected from collector). Routes malformed files to quarantine |

| Component | Responsibility |
|---|---|
| **Normalizer** | Validates each parsed event against the OCSF JSON Schema. Flags schema violations. Applies enrichment mappings (e.g., IP → GeoIP if available) |

### 4.2.4 Analysis Module

| Component | Responsibility |
|---|---|
| **Feature Extractor** | Computes statistical and contextual features from normalized events: IP entropy, login frequency, time-of-day deviation, byte volume anomaly, process execution rarity |
| **Rule Engine** | Evaluates feature-enriched events against Sigma-compatible detection rules loaded from PostgreSQL. Returns rule match alerts with severity and rule metadata |
| **AI Client** | HTTP client that sends feature vectors to the Python FastAPI AI Engine. Handles timeouts gracefully (fallback to rule-only). Implements `IAIClient` interface |

### 4.2.5 Correlation Module

| Component | Responsibility |
|---|---|
| **Incident Correlator** | Groups related alerts (from Rule Engine and AI Engine) by entity (user, host, IP), time window (configurable, default 15 min), and kill chain stage. Creates new incidents or merges alerts into existing open incidents |
| **Risk Scorer** | Computes composite risk score: `rule_weight × ML_confidence × asset_criticality`. Assigns severity level (Critical, High, Medium, Low, Informational). Handles missing factors with neutral defaults (1.0) |

### 4.2.6 Incidents Module

| Component | Responsibility |
|---|---|
| **Incident Service** | CRUD operations for incidents. Lifecycle management: Open → Investigating → Resolved → Closed. Assignment, notes, linking to constituent alerts and events |

### 4.2.7 Rules Module

| Component | Responsibility |
|---|---|
| **Rule Service** | CRUD operations for Sigma-compatible detection rules. Enable/disable without deletion. Import/export in YAML format. Rule testing against historical events (dry-run) |

### 4.2.8 Auth Module

| Component | Responsibility |
|---|---|
| **Auth Service** | Local username/password authentication with bcrypt. JWT token generation and validation (stored in Redis). RBAC enforcement (Admin, Security Engineer, SOC Analyst). Session management |
| **Audit Logger** | Records all user actions (login, rule changes, incident updates, config changes) as immutable audit entries in PostgreSQL |

### 4.2.9 Collector Monitoring Module

| Component | Responsibility |
|---|---|
| **Collector Status Service** | Reads heartbeat files from collector directory. Tracks: status (Online/Offline/Degraded), last heartbeat timestamp, files processed count, queue size (from `IProcessingQueue.getStatus()`), failed/quarantined file count, collector resource usage, last batch processed timestamp |

### 4.2.10 Dashboard Module

| Component | Responsibility |
|---|---|
| **Dashboard Service** | Aggregation queries for dashboard widgets: incident counts by severity, trend over time, top affected assets, recent incidents feed. Provides data for all dashboard visualizations |

### 4.2.11 Shared Kernel

| Component | Responsibility |
|---|---|
| **OCSF Schema Definitions** | TypeScript types/interfaces for all OCSF event categories used in the platform |
| **Error Handling** | Centralized error classes (DomainError, ApplicationError, InfrastructureError) with consistent structure |
| **Logger** | Structured JSON logging with correlation IDs, request tracing, and configurable log levels |
| **Configuration** | Environment-based configuration loading (12-Factor App). Validation on startup |

## 4.3 AI Engine (Python FastAPI) — Process 3

| Component | Responsibility |
|---|---|
| **Anomaly Detection Endpoint** | Receives feature vectors, runs Isolation Forest inference, returns anomaly scores |
| **Explainability Endpoint** | Computes SHAP values for a given prediction, returns feature importance rankings |
| **Threat Classification Endpoint** (optional) | Classifies events by threat category using Random Forest / XGBoost |
| **Feature Preprocessor** | Scales and encodes incoming features to match model training format |
| **Model Loader** | Loads serialized models (joblib/ONNX) at startup based on `models/config.json` version pointer |
| **Pydantic Schemas** | Request/response validation for all endpoints |

## 4.4 Frontend (Next.js) — Process 4

| Page | Responsibility |
|---|---|
| **Dashboard ( `/` )** | Real-time overview with incident counters, severity charts, timeline, top affected assets |

| Page | Responsibility |
|---|---|
| **Incidents ( `/incidents` )** | Incident list with filtering/sorting. Detail view with alerts, events, AI explanation, risk breakdown |
| **Investigation ( `/investigate` )** | Log search with time range, source, severity, keyword filters. Event timeline. Pivot to related incidents |
| **Rules ( `/rules` )** | Sigma rule YAML editor with syntax highlighting. Rule testing (dry-run). Import/export. Enable/disable |
| **AI Insights ( `/ai-insights` )** | SHAP explanation visualizations. Model confidence scores. Anomaly score distributions |
| **Collector Monitor ( `/collector` )** | Collector status, heartbeat, files processed, queue size, failed files, resource usage |
| **Settings ( `/settings` )** | User management (CRUD + roles). Data source configuration. Retention policies |
| **Audit Log ( `/audit` )** | Immutable record of all analyst and admin actions, filterable by user, action type, time range |

# 5. Technology Stack

## 5.1 Complete Technology Map

| Layer | Technology | Version Target | Purpose |
|---|---|---|---|
| **Collector** | C++ | C++20 | High-performance log collection and OCSF conversion |
| | CMake | 3.20+ | Cross-platform build system |
| | nlohmann/json | 3.x | JSON serialization/deserialization |
| | yaml-cpp | 0.7+ | YAML configuration parsing |
| | spdlog | 1.x | Fast structured logging |

| Layer | Technology | Version Target | Purpose |
|---|---|---|---|
| | Google Test | 1.x | Unit testing framework |
| **Backend** | Node.js | 20 LTS | JavaScript runtime |
| | Express | 4.x | HTTP framework |
| | TypeScript | 5.x | Type-safe JavaScript |
| | BullMQ | 5.x | Redis-backed job queue (behind IProcessingQueue) |
| | chokidar | 3.x | File system watcher |
| | pg (node-postgres) | 8.x | PostgreSQL client |
| | Mongoose | 8.x | MongoDB ODM |
| | ioredis | 5.x | Redis client |
| | jsonwebtoken | 9.x | JWT token management |
| | bcrypt | 5.x | Password hashing |
| | Joi or Zod | latest | Request validation |
| | ws | 8.x | WebSocket server |
| | Winston or Pino | latest | Structured logging |
| | Jest | 29.x | Unit/integration testing |
| **AI Engine** | Python | 3.11+ | ML runtime |
| | FastAPI | 0.100+ | Async REST framework |
| | scikit-learn | 1.x | Isolation Forest, Random Forest |
| | SHAP | 0.x | Model explainability |
| | XGBoost | 2.x | Optional threat classification |
| | Pydantic | 2.x | Request/response validation |
| | joblib | 1.x | Model serialization |
| | uvicorn | 0.x | ASGI server |

| Layer | Technology | Version Target | Purpose |
|---|---|---|---|
| | pytest | 8.x | Testing framework |
| **Frontend** | Next.js | 14+ | React SSR framework |
| | React | 18+ | UI library |
| | TypeScript | 5.x | Type safety |
| | Recharts or Chart.js | latest | Dashboard charts |
| | Zustand or React Context | latest | State management |
| | Axios or Fetch | latest | REST API client |
| | Monaco Editor | latest | YAML rule editor |
| **PostgreSQL** | PostgreSQL | 16 | Relational database |
| **MongoDB** | MongoDB | 7.x | Document database |
| **Redis** | Redis | 7.x | Cache, queue, pub/sub |
| **Deployment** | Docker | latest | Containerization |
| | Docker Compose | v2 | Multi-container orchestration |

## 5.2 Technology Rationale Matrix

| Decision | Chosen | Rejected Alternatives | Rationale |
|---|---|---|---|
| Collector language | C++ | Go, Rust | Maximum throughput for log parsing; zero-copy potential; existing team skill |
| Backend framework | Express (Node.js) | Fastify, NestJS, Koa | Mature ecosystem, largest community, minimal magic, easy to reason about |
| AI framework | FastAPI (Python) | Flask, Django REST | Async-native, auto OpenAPI docs, Pydantic validation, best-in-class for ML serving |

| Decision | Chosen | Rejected Alternatives | Rationale |
|---|---|---|---|
| Frontend framework | Next.js | Vite + React, Remix | SSR for fast initial load, App Router, API routes for BFF, industry standard |
| Relational DB | PostgreSQL | MySQL, SQLite | JSONB support, row-level security, advanced indexing, proven at scale |
| Document DB | MongoDB | CouchDB, DynamoDB | Flexible schema for heterogeneous logs, TTL indexes, text search, strong Node.js ecosystem |
| Cache/Queue | Redis + BullMQ | In-memory queue, RabbitMQ | BullMQ provides persistent queue with retry/DLQ; Redis provides pub/sub + cache; single dependency |

# 6. Communication Between Modules

## 6.1 Inter-Process Communication Matrix

| From | To | Protocol | Data Format | Pattern | Latency |
|---|---|---|---|---|---|
| **Collector** | **Backend** | File system (atomic write + watch) | OCSF JSON files (.json) | Async, fire-and-forget | Seconds (batch interval) |
| **Collector** | **Backend** | File system (heartbeat files) | JSON status file | Async, periodic | 30s intervals |
| **Backend** | **AI Engine** | HTTP REST | JSON request/response | Sync, request-reply | < 200ms target |
| **Frontend** | **Backend** | HTTP REST | JSON request/response | Sync, request-reply | < 500ms target |

| From | To | Protocol | Data Format | Pattern | Latency |
|---|---|---|---|---|---|
| **Backend** | **Frontend** | WebSocket (via Redis Pub/Sub) | JSON event push | Async, publish-subscribe | < 3s target |

## 6.2 Inter-Process Communication Diagram

graph LR subgraph "Collector (C++)" C[Collector Process] end subgraph "File System" FS["Collector Directory<br/>📁 batch_*.json<br/>📁 heartbeat.json"] end subgraph "Backend (Node.js)" B[Backend Process] end subgraph "AI Engine (Python)" AI[FastAPI Process] end subgraph "Frontend (Next.js)" FE[Next.js Process] end subgraph "Data Stores" PG[(PostgreSQL)] MDB[(MongoDB)] RD[(Redis)] end C ===>|"① OCSF JSON files<br/>(atomic write)"| FS FS ===>|"② File watch event<br/>(chokidar)"| B B <===>|"③ HTTP REST<br/>POST /api/v1/detect/*<br/>POST /api/v1/explain"| AI FE ===>|"④ HTTP REST<br/>GET/POST /api/v1/*"| B B -.->|"⑤ WebSocket push<br/>(via Redis Pub/Sub)"| FE B <===> PG & MDB & RD

## 6.3 Intra-Backend Module Communication

Inside the backend monolith, modules communicate through **dependency injection** and **interfaces** — never through direct class instantiation or shared mutable state.

graph TD subgraph "Module Communication Contracts" ING["Ingestion Module"] -->|"IProcessingQueue.enqueue()"| IPQ{{"IProcessingQueue"}} IPQ -->|"BullMQQueue.dequeue()"| WRK["Workers"] WRK -->|"IParser.parse()"| PAR{{"IParser"}} PAR -->|"INormalizer.normalize()"| NOR{{"INormalizer"}} NOR -->|"IFeatureExtractor.extract()"| FEX{{"IFeatureExtractor"}} FEX -->|"IRuleEngine.evaluate()"| RUL{{"IRuleEngine"}} FEX -->|"IAIClient.detect()"| AIC{{"IAIClient"}} RUL -->|"IIncidentCorrelator.correlate()"| COR{{"IIncidentCorrelator"}} AIC -->|"IIncidentCorrelator.correlate()"| COR COR -->|"IRiskScorer.score()"| RSC{{"IRiskScorer"}} RSC -->|"IIncidentRepository.save()"| IREP{{"IIncidentRepository"}} RSC -->|"ILogRepository.saveBatch()"| ILREP{{"ILogRepository"}} CTR["REST Controllers"] -->|"IIncidentService.findAll()"| ISVC{{"IIncidentService"}} CTR -->|"IRuleService.create()"| IRSVC{{"IRuleService"}} CTR -->|"ICollectorStatus.getStatus()"| ICSVC{{"ICollectorStatusService"}} end style IPQ fill:#3498db,color:#fff style PAR fill:#3498db,color:#fff style NOR fill:#3498db,color:#fff style FEX fill:#3498db,color:#fff style RUL fill:#3498db,color:#fff style AIC fill:#3498db,color:#fff style COR fill:#3498db,color:#fff style RSC fill:#3498db,color:#fff style IREP fill:#3498db,color:#fff style ILREP fill:#3498db,color:#fff style ISVC fill:#3498db,color:#fff style IRSVC fill:#3498db,color:#fff style ICSVC fill:#3498db,color:#fff

**Note**

Every diamond node (blue) in the diagram above is an **interface** defined in the domain layer. Concrete implementations live in the infrastructure layer and are injected at application startup. This is the **Dependency Inversion Principle** in action — the entire pipeline can be tested with mock implementations, and any component can be replaced without modifying its consumers.

# 6.4 API Contract Summary

## Backend REST API (Express)

| Method | Endpoint | Module | Description |
|---|---|---|---|
| `POST` | `/api/v1/auth/login` | Auth | Authenticate user, return JWT |
| `POST` | `/api/v1/auth/logout` | Auth | Invalidate session |
| `GET` | `/api/v1/incidents` | Incidents | List incidents (paginated, filterable) |
| `GET` | `/api/v1/incidents/:id` | Incidents | Get incident detail with alerts and events |
| `PUT` | `/api/v1/incidents/:id` | Incidents | Update incident (status, assignee, notes) |
| `GET` | `/api/v1/events` | Dashboard | Search normalized events (paginated, filterable) |
| `GET` | `/api/v1/events/:id` | Dashboard | Get single OCSF event detail |
| `GET` | `/api/v1/rules` | Rules | List detection rules |
| `POST` | `/api/v1/rules` | Rules | Create new Sigma rule |
| `PUT` | `/api/v1/rules/:id` | Rules | Update rule |
| `DELETE` | `/api/v1/rules/:id` | Rules | Delete rule |
| `POST` | `/api/v1/rules/:id/test` | Rules | Dry-run rule against historical events |
| `GET` | `/api/v1/dashboard/summary` | Dashboard | Aggregated dashboard data |
| `GET` | `/api/v1/collector/status` | Collector Monitoring | Current collector health and metrics |

| Method | Endpoint | Module | Description |
|---|---|---|---|
| GET | /api/v1/users | Auth | List users (Admin only) |
| POST | /api/v1/users | Auth | Create user (Admin only) |
| PUT | /api/v1/users/:id | Auth | Update user (Admin only) |
| GET | /api/v1/audit | Auth | Audit log (paginated, filterable) |
| GET | /api/v1/health | Shared | Health check |
| WS | /ws | Dashboard | WebSocket for real-time updates |

## AI Engine REST API (FastAPI)

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/v1/detect/anomaly | Isolation Forest anomaly detection |
| POST | /api/v1/explain | SHAP explanation for anomaly score |
| POST | /api/v1/classify/threat | Optional threat classification |
| GET | /api/v1/health | Health check |

> **This HLD is governed by [ADR-001](file:///d:/AI%20SIEM/docs/architecture/decisions/ADR-001-modular-monolith.md) and implements the requirements defined in [SRS-001](file:///d:/AI%20SIEM/docs/SRS.md).**
>
> **Document Version**: 1.0
> **Last Updated**: 2026-07-18